

Performance Analysis of Sorting Algorithms

Student Name: Ajaya Kumar Patel

Roll Number: CRS2503

Course: Cryptographic and Security Implementations

August 3, 2026

Contents

1	Introduction	2
2	Experimental Setup	2
3	Analysis 1: Normalized Comparisons and Swaps	2
4	Analysis 2: CPU Performance Measurement	3
5	Analysis 3: Size vs Time Analysis	3
6	Conclusion	4
7	References	5

1 Introduction

This report presents the performance comparison of three sorting algorithms: Merge Sort, Quick Sort, and Heap Sort.

The algorithms were implemented in C and tested using randomly generated integer arrays. The comparison is based on operation count, CPU performance, and execution time for different input sizes.

2 Experimental Setup

The experiment was performed using:

- Array size: 10000 elements
- Number of iterations: 100
- Input data: Randomly generated integers
- CPU measurement: CPU cycle counter using `_rdtscp()`
- Time measurement: `clock_gettime()`
- Compiler optimization: GCC with `-O3` optimization flag

3 Analysis 1: Normalized Comparisons and Swaps

To compare the number of operations independent of input size, the number of comparisons and swaps were normalized using:

$$\frac{\text{Operation Count}}{n \log_2 n}$$

The results are shown below.

Algorithm	Normalized Comparisons	Normalized Swaps
Merge Sort	0.9069	0.0000
Quick Sort	1.2280	0.6834
Heap Sort	1.7716	0.9351

Table 1: Normalized comparison and swap count

Merge Sort performs only comparisons because it does not use swapping during merging. Quick Sort and Heap Sort perform swaps during rearrangement of elements.

4 Analysis 2: CPU Performance Measurement

The execution performance was measured using average CPU time, average CPU cycles, cycles per byte, and cycles per bit. The measurements were performed on randomly generated arrays of size $n = 10000$ elements, with 100 iterations to obtain the average values.

Algorithm	Average Time (ns)	CPU Cycles	Cycles/Byte	Cycles/Bit
Merge Sort	2686647.70	6705895.10	167.6474	20.955922
Quick Sort	615540.91	1536386.48	38.4097	4.801208
Heap Sort	742380.79	1852987.90	46.3247	5.790587

Table 2: CPU performance comparison

From the measurements, Quick Sort required the lowest execution time and CPU cycles for the tested random input. Heap Sort performed better than Merge Sort in execution time but required more operations than Quick Sort.

5 Analysis 3: Size vs Time Analysis

The execution time was measured for different input sizes ranging from 1000 to 10000 elements.

Array Size	Merge Sort	Quick Sort	Heap Sort
1000	141514.16	69413.70	120055.58
2000	367711.12	243117.04	385176.04
3000	579173.90	404994.80	620451.60
4000	662458.42	389725.44	576239.68
5000	865603.84	602508.68	772717.18
6000	1013812.02	610968.28	908320.90
7000	1061139.56	735228.04	1103654.88
8000	1217622.82	846908.10	1307778.38
9000	1367534.30	1012618.42	1505093.74
10000	1972533.08	1404355.32	1624419.70

Table 3: Execution time for different input sizes

The following graph shows the change in execution time with increasing array size.

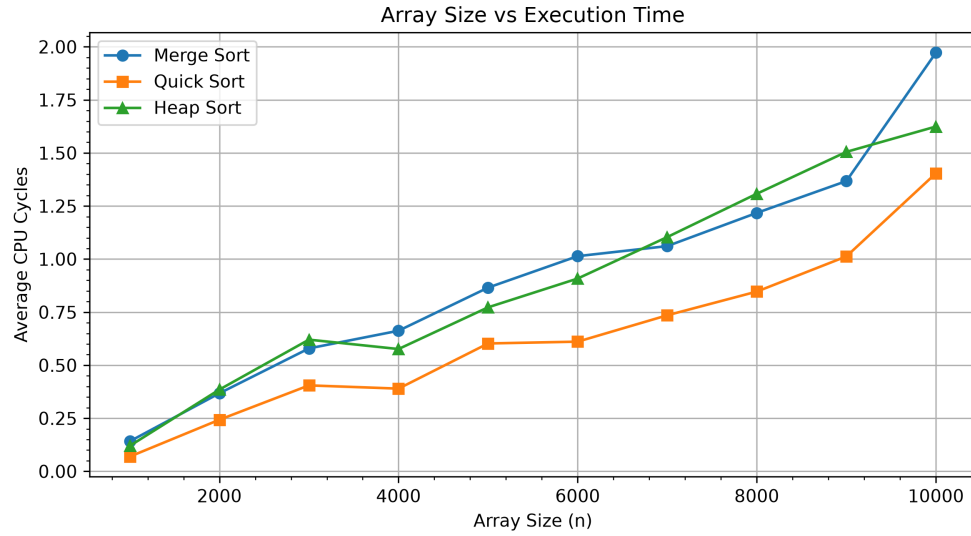


Figure 1: Array Size vs Execution Time

6 Conclusion

The experiment shows that all three algorithms follow approximately $O(n \log n)$ behavior.

For the tested random input:

- Quick Sort achieved the lowest execution time and CPU cycles.
- Heap Sort required more comparisons and swaps than Quick Sort.
- Merge Sort showed higher execution time because of additional merging operations and memory usage.

Overall, Quick Sort provided the best performance for the tested dataset.

7 References

1. OpenAI. *ChatGPT (GPT-5.5)*. Used for benchmarking methodology, implementation guidance, and code review. <https://chatgpt.com/>
2. GeeksforGeeks. *Merge Sort*. <https://www.geeksforgeeks.org/merge-sort/>
3. GeeksforGeeks. *Quick Sort*. <https://www.geeksforgeeks.org/quick-sort/>
4. GeeksforGeeks. *Heap Sort*. <https://www.geeksforgeeks.org/heap-sort/>